



EPHEC

Avenue du Ciseau 15 – 1348 Louvain-la-Neuve

Bachelier en Informatique,
orientation « Technologies de l'informatique »

T3032 – Traitement des signaux et données

Énoncés des laboratoires

Arnaud Dewulf et Stéphanie Guérit

Année académique 2024-2025

Auteur du syllabus d'origine en Matlab : Pierre Ernotte
Mises à jour/en forme : Virginie Van den Schriek, Arnaud Dewulf et Stéphanie Guérit

Table des matières

I	Les signaux et Python	5
1	Introduction au calcul matriciel avec Python	7
1.1	Prise en main	7
1.2	Vecteurs et matrices	8
1.3	Programmation en Python	10
1.4	Éléments de calcul matriciel	11
2	Représentation des signaux	13
2.1	Visualisation de signaux 1D et d'images	13
2.2	Représentation temporelle de signaux périodiques	15
II	Traitement des signaux	17
3	Représentation fréquentielle de signaux 1D	19
3.1	Séries de Fourier	19
3.2	Transformée de Fourier	20
3.3	Filtrage	23
4	Traitement d'images	25
4.1	Ouverture, affichage et types d'images	25
4.2	Opérations arithmétiques sur une image	26
4.3	Opérations sur une image binaire	29
6	Vademecum Python	31
6.1	Introduction au calcul matriciel avec Python	31
6.2	Représentation des signaux	33
6.3	Représentation fréquentielle des signaux 1D	34
6.4	Traitement d'images	35
III	Introduction au machine learning	39
7	<i>Clustering</i>	41
7.1	Jeu de données : les iris de Fisher (1936)	41
7.2	Les ingrédients du <i>clustering</i>	42
7.3	Algorithme <i>k</i> -means	42
8	Classification binaire	45
8.1	Les ingrédients de la classification	45
8.2	Jeu de données : cancer du sein	45
8.3	Régression linéaire	46
8.4	Comparaison de modèles	47

Thème I

Les signaux et Python

Partie 1

Introduction au calcul matriciel avec Python

Les trois packages qui seront utilisés dans cette section (et également dans les suivantes) sont repris ci-dessous. Pensez à ajouter ces commandes au début de votre code.

```
import numpy as np
import sounddevice as sd
from scipy.io.wavfile import read
```

1.1 Prise en main

Exercice 1.1 Exécutez les deux commandes suivantes :

```
1
2;
```

Exécutez ensuite les quatre commandes suivantes. Que pouvez-vous en conclure ?

```
1
2;
print(3)
print(4)
```

Exercice 1.2 Exécutez `help(np.angle)`.

Exercice 1.3 Exécutez les deux commandes suivantes. Que pouvez-vous en conclure ?

```
1+2j
1+2i
```

Exécutez ensuite les deux lignes suivantes. Qu'observez-vous ?

```
1+1j
1+j
```

Exercice 1.4 Définissez deux variables `a` et `b` de la façon suivante :

```
a = 1+1j
b = 1-1j
```

Exécutez ensuite les instructions suivantes pour calculer le produit des deux nombres, le complexe conjugué de `a` ainsi que le module (amplitude) et l'argument (phase) de `a`.

```
print(a*b)
print(a.conjugate())      # donne le complexe conjugué de a
print(abs(a))             # donne le module de a
print(np.angle(a))        # donne l'argument de a en radians
```

Notez qu'on utilise le symbole `#` pour commenter.

Exercice 1.5 Calculez le module et l'argument des nombres complexes suivants :

- a) $2 + 5j$ b) $2 - 5j$ c) 2 d) $5j$

Exercice 1.6 Comparez et commentez les résultats obtenus en calculant `np.cos(180)`, `np.cos(np.radians(180))` et `np.cos(np.deg2rad(180))`.

Les fonctions `np.rad2deg` et `np.degrees` permettent quant à elles de convertir en degrés un angle exprimé en radians.

Exercice 1.7 Calculez les parties réelle et imaginaire des nombres complexes suivants :

- a) $5\text{cjs}(\alpha)$ avec $\alpha = 25^\circ$ (*rappel* : $\text{cjs}(\alpha) = \cos \alpha + j \sin \alpha$)
 b) $5e^{j\alpha}$ avec $\alpha = 25^\circ$ (*conseil* : utilisez la fonction `np.exp`)

Vérifiez que les valeurs obtenues aux points (a) et (b) sont égales. Ensuite, faites de même avec $7\text{cjs}(167)$.

1.2 Vecteurs et matrices

Exercice 1.8 Il y a plusieurs façons de créer un vecteur de type `numpy.ndarray` avec Python. Exécutez les instructions suivantes :

```
t1 = np.arange(1, 11)           # définit un vecteur allant de 1 à 10
t2 = np.arange(1, 10.1, 0.1)    # définit un vecteur allant de 1 à 10 avec
                                # un pas de 0.1
t3 = np.linspace(1, 10, 100)    # définit un vecteur contenant 100 entrées
                                # réparties uniformément entre 1 et 10
t4 = np.array([1, 3, 5, 2, -1]) # définit le vecteur en spécifiant chaque
                                # entrée
```

La deuxième instruction permet de définir le pas, c'est-à-dire la différence entre les valeurs de deux entrées successives du vecteur. La troisième instruction permet plutôt de spécifier le nombre d'entrées que l'on souhaite dans le vecteur.

Exercice 1.9 Soit `t1 = np.arange(1, 11)` et `t2 = np.arange(1, 10.1, 0.1)`. Exécutez l'instruction suivante :

```
%whos
```

Interprétez le résultat obtenu. Constatez que `t1` est un vecteur contenant 10 éléments de type `'int32'` et `t2` est un vecteur contenant 91 éléments de type `'float64'`.

Exercice 1.10 Soit `t = np.arange(11, 21)`. Vous pouvez visualiser une ou plusieurs entrées de `t` en procédant de différentes manières :

- a) `t[4]` désigne le cinquième élément du vecteur `t` ;
- b) `t[-1]` désigne le dernier élément du vecteur `t` ;
- c) `t[-2]` désigne l'avant-dernier élément du vecteur `t` ;
- d) `t[:5]` désigne les cinq premiers éléments du vecteur `t` ;
- e) `t[7:]` désigne les trois derniers éléments du vecteur `t`.

Important

En Python (comme en C ou en Java), les indices d'un vecteur commencent à 0.

Exercice 1.11 Soit `t = np.arange(1, 11)`. Exécutez `print(len(t))`, `print(t.shape)`, `print(np.size(t))` et `print(np.max(t.shape))`. Interprétez les résultats au moyen de l'aide de ces fonctions.

Exercice 1.12 Soit `x = np.array([1, 3, 7, 9, 20, 40])`. Que valent `x[1]`, `x[1:4]` et `x[1:6:2]` ?

Exercice 1.13 Exécutez les instructions suivantes :

```
a = np.array([1, 2, 3])
b = np.array([[1], [2], [3]])
```

Quelle(s) différence(s) observez-vous entre les deux cas ? Lequel représente un vecteur colonne ? et un vecteur ligne ?

Exercice 1.14 Une matrice est un tableau rectangulaire possédant des lignes et des colonnes. Chaque entrée du tableau contient un nombre. On peut définir une matrice en spécifiant chaque entrée de celle-ci, par exemple

```
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

Exécutez cette commande. Ensuite, affichez l'élément $m_{2,3}$ de la matrice `m`, c'est-à-dire l'élément localisé à la deuxième ligne et la troisième colonne.

On peut également afficher tout une ligne ou toute une colonne d'une matrice. Exécutez les deux instructions suivantes.

```
print(m[1, :])
print(m[:, 1])
```

Laquelle permet d'afficher la ligne 2 ? Laquelle permet d'afficher la colonne 2 ?

Exercice 1.15 On peut concaténer plusieurs vecteurs ou matrices pour autant que leurs dimensions soient compatibles. Exécutez les instructions suivantes et observez les vecteurs ou matrices `c`, `d` et `e` résultants.

```

a = np.array([1, 2, 3])
b = np.array([[1], [2], [3]])
c = np.array([a, a])
d = np.array([b.flatten(), b.flatten()])
e = d.T

```

Exercice 1.16 Un signal audio est un vecteur : l'entrée d'indice i correspond à l'amplitude du signal audio au temps discret i .

- a) Ouvrez le fichier `chirac1.wav` au moyen de l'instruction `read`.

```
Fs, s = read('chirac1.wav')
```

À quoi correspondent `s` et `Fs` ? Quelle est la valeur de `Fs` ? Quelles sont les valeurs maximale et minimale de `s` ?

- b) Écoutez ce son au moyen de l'instruction `sd.play(s)`. Expliquez pourquoi celle-ci ne donne pas un résultat correct. Adaptez-la pour écouter le son correctement.
- c) Définissez un vecteur qui prend les 100 premiers éléments de `s`. Écoutez le résultat. Pourquoi n'entendez-vous rien ?
- d) Définissez un vecteur qui prend les éléments 10000 à 20000 de `s` et divise l'amplitude originale par 4 (attention, utilisez une division entière!). Écoutez le résultat.
- e) Définissez un vecteur qui est la concaténation du vecteur d'indices 0 à 10000, du vecteur d'indices 10001 à 20000 et du vecteur d'indice 20001 jusqu'à l'indice du dernier élément de `s`. Écoutez le résultat obtenu.

1.3 Programmation en Python

Voici un rappel de la syntaxe des boucles principales :

- boucle `for` :

```

for i in range(1,11):
    instructions

```

- boucle `while` :

```

while condition:
    instructions

```

- condition :

```

if condition1:
    instructions
elif condition2:
    instructions
else:
    instructions

```

- Pour les conditions, on utilisera les opérateurs logiques `and` (et), `or` (ou) et `not` (non) ainsi que `<` `<=` `>` `>=` `==` `!=`. Les deux dernières expressions sont utilisées pour *est égal à* ou *n'est pas égal à*.

Exercice 1.17 Écrivez, au moyen d'une boucle, un programme qui affiche le carré des nombres de 10 à 20.

Exercice 1.18 Écrivez, au moyen d'une boucle, un programme qui donne le sinus d'un angle de 0° à 360° avec un pas de 10° .

Important

Les deux derniers exercices sont des cas typiques où les boucles sont à éviter à cause de leur lenteur. Il sera de loin préférable d'utiliser des outils vectoriels comme la fonction « `np.square` » que nous verrons à la section 1.4.

1.4 Éléments de calcul matriciel

Exercice 1.19 Créez la matrice `M = np.array([[1, 2], [3, 4]])` et la matrice `N` dont la première ligne vaut `2 1` et la deuxième ligne vaut `1 -1`. On pourra utiliser `N[1, :] = [2, 1]`.

- Calculez `M @ N` (ou `np.matmul(M, N)`). Vérifiez théoriquement le résultat obtenu.
- Calculez `N @ M` et constatez la différence avec le résultat précédent.

Exercice 1.20 Créez la matrice `O` constituée de la colonne `-2 1`. En utilisant la matrice `M` de l'exercice 1.19, calculez les matrices `M @ O` et `O @ M`. Vérifiez théoriquement le résultat obtenu.

Exercice 1.21 Créez une matrice `M` de taille 4×4 grâce à l'instruction `np.ones`. Remplacez la deuxième colonne de cette matrice par des 2 et remplacez la troisième ligne du résultat par des 3. Affichez le résultat. Créez ensuite une matrice `I` de taille 4×4 contenant des 1 sur la diagonale principale et des 0 partout ailleurs. Cette matrice est une *matrice identité*. Vous pouvez utiliser la fonction `np.eye` ou la construire à partir de la fonction `np.zeros`.

- Calculez `M @ I` et `I @ M`. Que pouvez-vous en conclure ?
- Calculez `F = M + I` puis `G = np.linalg.inv(F)` (*inverse* de `F`) et `F @ G`. Que pouvez-vous en conclure ?
- La matrice `M` possède-t-elle une inverse ?

Exercice 1.22 En plus de la matrice `M` définie à l'exercice 1.21, créez une matrice `C` de 2 lignes et 4 colonnes, `C = np.array([[0, 1, 2, 3], [3, 4, 5, 6]])`.

- a) Calculez $M @ C$ et $C @ M$. Pourquoi obtient-on une erreur avec $M @ C$?
- b) Calculez $C.T$ (*transposée* de C) et $M @ C.T$. Pourquoi n'obtient-on pas d'erreur en calculant $M.T @ C.T$?

Exercice 1.23 Soit la matrice M obtenue à l'exercice 1.21.

- a) Calculez `np.linalg.matrix_power(M, 2)` et $M @ M$.
- b) Calculez `np.square(M)` et $M * M$ et comparez au résultat du point précédent.

Exercice 1.24 L'opération $*$, par exemple $x * y$, donne une multiplication terme à terme des deux facteurs. Si l'un d'eux est un scalaire, il multiplie tous les termes de l'autre sinon x et y doivent avoir la même dimension. En utilisant les matrices définies dans les exercices précédents,

- a) calculez $M * I$, $2 * M$ et $C * M$ et expliquez vos réponses ;
- b) interprétez `len(variable)` et `variable.shape` pour M , I et C .

Exercice 1.25 Écrivez un programme qui affiche le carré des nombres de 10 à 20 sans utiliser de boucle.

Important

Les calculs faisant intervenir des boucles sont souvent lents. Il est plus efficace d'utiliser des outils de calcul matriciel.

Comparez les programmes suivants qui affichent tous deux le sinus des angles de 0 à 1000000 :

```
import time

# Programme 1
t1_Start = time.time()
for i in range(1000001):
    np.sin(i)
t1_End = time.time()

print("Temps d'exécution du programme 1 : ", t1_End-t1_Start)

# Programme 2
t2_Start = time.time()
np.sin(np.arange(1000001))
t2_End = time.time()

print("Temps d'exécution du programme 2 : ", t2_End-t2_Start)
```

Le programme 2 a un output équivalent au premier mais s'exécute plus rapidement.

Partie 2

Représentation des signaux

Dans cette partie, nous abordons la visualisation de fonctions et d'images avec la librairie Matplotlib. Pensez à l'importer dans votre code :

```
import matplotlib.pyplot as plt
```

Ensuite, une série d'exercices permettront de vous familiariser avec les signaux périodiques 1D que nous utiliserons majoritairement dans le reste du cours.

2.1 Visualisation de signaux 1D et d'images

La librairie Matplotlib permet d'afficher facilement des fonctions à une ou deux variables. Lisez et testez le code ci-dessous. Identifiez l'effet de chaque ligne sur la figure produite.

```
t = np.arange(0, 2.01, 0.01)
y = 0.5 * np.sin(t)

fig, ax = plt.subplots()
ax.plot(t, y)
ax.set_title('Graphe de la fonction sinus')
ax.set_xlabel('t [s]')
ax.set_ylabel('sin(t)')
ax.grid(True)

plt.show()
```

Quelques remarques :

- l'instruction `plt.show()` permet d'afficher toutes les figures ouvertes ;
- l'instruction `plt.subplots(a, b)` permet de dessiner plusieurs graphiques sur une même figure. Par exemple, `plt.subplots(2, 3)` indique que la figure sera divisée en une grille de 2×3 sous-figures ;
- l'instruction `plt.stem(t, y)` (ou, dans l'exemple ci-dessus, `ax.stem(t, y)`) permet d'afficher les données (t, y) sans les relier ;
- l'instruction `plt.close('all')` permet d'effacer toutes les fenêtres graphiques que vous avez créées. Pour ne fermer que la figure `fig`, utilisez `plt.close(fig)`.

Exercice 2.1 Écrivez un script qui affiche, sur quatre sous-figures placées l'une en dessous de l'autre, la fonction $\sin(t)$:

- a) au moyen de la fonction `plot` avec `t` variant de 0 à 4 avec 21 points ;

- b) au moyen de la fonction `plot` avec `t` variant de 0 à 4 avec 9 points ;
- c) au moyen de la fonction `stem` avec `t` variant de 0 à 4 avec 21 points ;
- d) au moyen de la fonction `stem` avec `t` variant de 0 à 4 avec 9 points.

N'hésitez pas à vous aider de cette démo disponible sur le site de la librairie : https://matplotlib.org/stable/gallery/subplots_axes_and_figures/subplots_demo.html.

Exercice 2.2 Soit `t = np.arange(0, 10.1, 0.1)` et `y = t * t`. Quelle différence existe-il entre `plt.plot(t, y)` et `plt.plot(y)` ?

Exercice 2.3 Exécutez l'instruction suivante et observez le résultat obtenu :

```
plt.plot(t, np.sin(t), t, np.cos(t))
```

Exercice 2.4 Une image en niveaux de gris (8 bits) est une matrice de nombres entiers compris entre 0 et 255.

- a) Ouvrez l'image `Lenna.bmp` au moyen de la fonction `plt.imread`.
- b) Visualisez l'image au moyen de la fonction `plt.imshow` suivie de `plt.show()`. Comment afficher l'image en niveaux de gris ?
- c) Définissez une matrice contenant uniquement un de ses yeux.
- d) Calculez la moyenne de la valeur de cette sélection à l'aide de la fonction `np.mean`.
- e) Ses yeux sont-ils plutôt clairs ou foncés ?

Exercice 2.5 Il est aussi possible de visualiser un tableau de données grâce à l'instruction `imshow`. Visualisez un tableau de 5 lignes et 5 colonnes rempli de nombres aléatoires (vous pouvez utiliser pour cela `np.random.rand`, `np.random.randint` ou `np.random.randn`).

Exercice 2.6 Créez une matrice `A` de taille 10×10 composée de nombres aléatoires compris entre 0 et 1 (voir exercice 2.5). Visualisez cette matrice comme une image au moyen des instructions suivantes :

```
fig, ax1 = plt.subplots()
pos = ax1.imshow(A)
fig.colorbar(pos, ax=ax1)
plt.show()
```

En adaptant l'argument `cmap` de `ax1.imshow`, choisissez la colormap `'jet'`.

Exercice 2.7 En utilisant la matrice `A` définie à l'exercice 2.6, affichez sur trois sous-figures disposées en colonnes les `plot` des

- éléments de la première ligne ;
- éléments de la première colonne ;
- éléments de la diagonale principale (*conseil* : utilisez la fonction `np.diag`).

Pour que les trois sous-figures partagent le même axe des ordonnées, ajoutez l'argument `sharey=True` à `plt.subplots`.

2.2 Représentation temporelle de signaux périodiques

À partir de cette section, nous utiliserons une nouvelle librairie spécifique au traitement des signaux, SciPy. Pensez à ajouter au début de votre code la ligne suivante :

```
from scipy import signal
```

Exercice 2.8 Écrivez un code qui crée un signal sinusoïdal de 440 Hz, d'une amplitude de 0,5 et d'une durée de 4 s avec une fréquence d'échantillonnage de 22 kHz.

Indication : si la fréquence d'échantillonnage est `fe`, le pas entre deux échantillons est `Te = 1/fe`. On aura alors `t = np.arange(0, 4 + Te, Te)`.

Écoutez (à l'aide de l'instruction `sd.play`) et visualisez ce signal. Pourquoi n'est-il pas possible de le visualiser correctement ? Modifiez le signal pour que la visualisation soit possible, par exemple en se limitant à quatre périodes. Écoutez le nouveau signal. Pourquoi n'entendez-vous rien ?

Pour éviter de modifier le signal, vous pouvez aussi, lors de la visualisation, limiter l'intervalle de valeurs affichées en ajoutant `plt.xlim((0, 0.05))` avant `plt.show()`. L'axe x affichera uniquement le graphe pour t allant de 0 à 0,05 s.

Exercice 2.9 Écrivez un programme qui crée un signal qui est la somme d'un signal sinusoïdal de 440 Hz et d'un signal de 445 Hz, tous deux d'une durée de 4 s avec une fréquence d'échantillonnage de 22 kHz. Visualisez et écoutez le battement.

Exercice 2.10 Réalisez à nouveau l'exercice 2.9 mais cette fois avec deux sinus dont les fréquences sont 440 et 441 Hz.

Exercice 2.11 Sachant que le « La » de la troisième octave a une fréquence de 440 Hz et que l'on obtient la note suivante d'une gamme en multipliant la note précédente par $2^{1/6}$, créez une succession de 18 notes¹ au moyen d'une boucle `for`. Choisissez une fréquence d'échantillonnage de 11025 Hz. Écoutez le résultat obtenu.

Exercice 2.12 Créez un signal bruité de 440 Hz d'une durée de 4 s. Pour ajouter du bruit à votre signal sinusoïdal original, utilisez par exemple la fonction `np.random.rand`. Écoutez votre résultat.

1. La gamme n'est pas complète puisqu'elle ne contient que six notes au lieu de sept. Elle ne tient en effet pas compte des demi-tons présents dans une gamme musicale réelle.

Exercice 2.13 Créez une oscillation amortie dont la fréquence est de 440 Hz et la durée est 2 s. Choisissez un facteur d'amortissement adéquat. Visualisez et écoutez le résultat.

Exercice 2.14 Écrivez un programme pour tester les signaux obtenus avec les fonctions suivantes et leurs différents paramètres : `np.sinc`, `signal.square`, `signal.sawtooth` et `signal.chirp`. Écoutez et affichez le résultat pour chacun d'eux.

Thème II

Traitement des signaux

Partie 3

Représentation fréquentielle de signaux 1D

Les packages à ajouter en plus des précédents pour cette partie sur la représentation fréquentielle des signaux sont les suivants :

```
from numpy import fft
```

3.1 Séries de Fourier

Exercice 3.1 Un signal carré antisymétrique s de fréquence F_0 est périodique et peut donc être représenté par une série de Fourier,

$$s(t) = \sum_{n=0}^{\infty} \frac{1}{2n+1} \sin(2\pi(2n+1)F_0 t).$$

Ce signal est une somme de signaux sinusoïdaux de fréquences $(2n+1)F_0$ (avec $n \in \mathbb{N}$) et d'amplitude $\frac{1}{2n+1}$.

Écrivez un petit programme basé sur une boucle qui permet de dessiner successivement les premiers termes de cette série (d'abord seulement le premier terme $n = 0$, ensuite les deux premiers, jusqu'aux 6 premiers termes). Considérez $F_0 = 1$ Hz et $\mathbf{t} = \mathbf{np.arange}(0, 10.01, 0.01)$.

Exercice 3.2 Sur base du code de l'exercice 3.1, constatez que plus n augmente, plus les signaux obtenus s'approchent d'un signal carré. Visualisez le spectre de ces signaux en exécutant les lignes suivantes :

```
Fe = 100
f = fft.fftfreq(len(t), 1/Fe) # Vecteur des fréquences
TF = fft.fft(s)               # Transformée de Fourier

fig, ax = plt.subplots()
ax.plot(f, np.abs(TF))
ax.set_title(f'Spectre de l\'approximation de s avec {Nb_termes} termes')
ax.set_xlabel('f [Hz]')
ax.set_ylabel('Amplitude de la TF')

plt.tight_layout()
plt.show()
```

Exercice 3.3 Représentez le spectre bilatéral du signal $3 + 2 \cos(600t) - 3 \sin(2\pi 20t)$.

Exercice 3.4 Calculez théoriquement les expressions des spectres des fonctions suivantes et dessinez-les (représentation cartésienne et représentation polaire) :

- a) $\sin(2\pi t)$
- b) $\cos(2\pi t) - \sin(2\pi t)$
- c) $\sin(2\pi t + \frac{\pi}{4})$
- d) $\cos(2\pi t) + \cos(2\pi t + \frac{\pi}{2})$
- e) $\cos(2\pi t) + 0,5 \cos(2\pi 2t) + 0,25 \cos(2\pi 4t + \frac{\pi}{4})$
- f) $\cos(2\pi t) \cos(2\pi 440t)$

Astuce

Rappelez-vous que la transformée de Fourier est une *opération linéaire*, c'est-à-dire que la transformée de Fourier d'une somme de signaux est égale à la somme des transformées de Fourier de chaque signal pris séparément.

Astuce

Formules utiles :

$$\sin(a + b) = \sin a \cos b + \sin b \cos a$$

$$\cos(a + b) = \cos a \cos b - \sin a \sin b$$

$$2 \cos a \cos b = \cos(a + b) + \cos(a - b)$$

et

$$\cos a = \frac{e^{ja} + e^{-ja}}{2}, \quad \sin a = \frac{e^{ja} - e^{-ja}}{2j}.$$

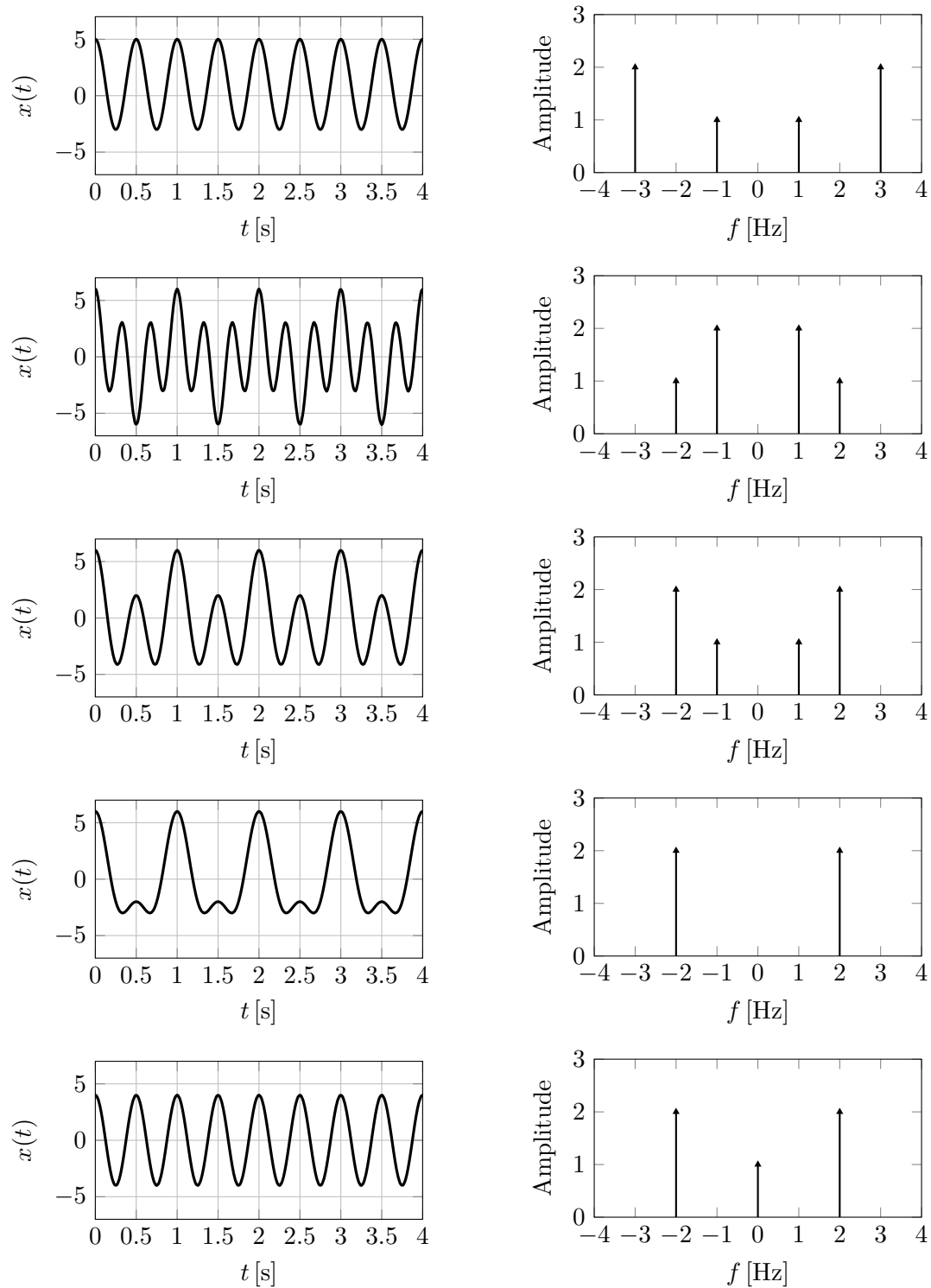
Exercice 3.5 Dans le Tableau 3.1, faites correspondre chaque signal à son spectre.

3.2 Transformée de Fourier

Exercice 3.6 Esquissez à la main le spectre des fonctions suivantes :

- fonction porte d'une durée de 4 s ;
- signal sinusoïdal de 10 Hz et d'une durée de 4 s ;
- signal sinusoïdal de 440 Hz et d'une durée de 4 s ;
- peigne de Dirac d'une période de 1 ms.

Exercice 3.7 Un signal sinusoïdal a une durée de 4 périodes. Théoriquement, que peut-on dire de « sa largeur de bande » ?

**Tableau 3.1** – Signaux et spectres de l'exercice 3.5. Source : [1].

3.2.1 Visualisation du spectre

Dans cette section, vous allez voir les instructions qui permettent de visualiser le spectre d'un signal en utilisant principalement deux fonctions : `fft.fft` et `fft.fftshift`.

Voici un exemple de programme permettant d'afficher la transformée de Fourier d'un sinus de 440 Hz, d'une durée de 2 s et échantillonné à 1000 Hz. L'axe des fréquences s'étend de $-F_e/2$ à $F_e/2$.

```
Fe = 1000
Te = 1/Fe
duree = 2

t = np.arange(0, duree+Te, Te)
s = np.sin(2*np.pi*440*t)

f = fft.fftfreq(len(t), 1/Fe) # Vecteur des fréquences
TF = fft.fft(s)               # Transformée de Fourier

plt.plot(f, np.abs(TF)) # On affiche uniquement le module de la TF
plt.xlabel('f [Hz]')
plt.ylabel('Amplitude de la TF')
plt.title('Transformée de Fourier du signal s')
plt.show()
```

Exercice 3.8 Écrivez un programme qui affiche le même spectre que celui de l'exemple ci-dessus mais pour un axe des fréquences qui s'étend de 0 à $F_e/2$.

Exercice 3.9 Affichez la FFT d'un signal sinusoïdal de 440 Hz d'une durée de 2 s échantillonné à 2 kHz. Affichez le spectre pour des fréquences variant de 0 à $F_e/2$.

Exercice 3.10 Affichez la FFT d'un signal d'une durée de 2 s constitué de la somme d'un cosinus de 50 Hz d'amplitude 2 et d'un cosinus de 440 Hz d'amplitude 1. Fixez la fréquence d'échantillonnage à $F_e = 2000$ Hz.

Exercice 3.11 Affichez la FFT du produit d'un cosinus de 50 Hz d'amplitude 2 avec un cosinus de 440 Hz d'amplitude 1. Les deux signaux ont une durée de 2 s et une fréquence d'échantillonnage $F_e = 2000$ Hz.

Exercice 3.12 Affichez la FFT de l'autocorrélation d'un signal sinusoïdal de 440 Hz d'une durée de 2 s échantillonné à 2 kHz. Affichez le spectre pour des fréquences variant de 0 à $F_e/2$ et avec une échelle en dB pour l'axe des ordonnées.

Représentez ensuite la *densité spectrale de puissance* (PSD) de ce signal sinusoïdal, en dB¹. *Indication* : si la transformée de Fourier d'un signal est $S(f)$, sa PSD est donnée par $20 \log S(f)$.

1. En dBm, on prendra 1 mW et une impédance de 50Ω comme référence.

3.3 Filtrage

Dans la section précédente, vous avez vu comment afficher le contenu fréquentiel d'un signal. À présent, nous allons voir comment *concevoir des filtres* qui permettent de *modifier* ce contenu fréquentiel. En plus des librairies habituelles (`numpy` et `matplotlib.pyplot`), nous utiliserons également pour les exercices suivants :

```
import sounddevice as sd
from scipy import signal
from scipy.io.wavfile import read
```

Exercice 3.13 Un filtre de Butterworth d'ordre 1 atténue un signal de 20 dB par décade. Écoutez un signal sinusoïdal de 440 Hz échantillonné à 2000 Hz. Grâce à la fonction `signal.butter`, concevez un filtre de Butterworth d'ordre 1 dont la fréquence de coupure est choisie pour obtenir une atténuation de -20 dB à 440 Hz. Filtrez le signal original grâce à la fonction `signal.filtfilt` et écoutez le signal filtré.

Exercice 3.14 En écoutant le son `alien_bruit1.wav`, on remarque que la chanson contient un bruit parasite dont on aimerait se débarrasser.

- Isolez la chanson de manière à ne pas déformer les voix lors du filtrage.
- Affichez le spectre de cette chanson bruitée et déduisez-en la fréquence du signal parasite.
- Créez un filtre (*e.g.*, un filtre de Butterworth d'ordre 2), appliquez-le au son corrompu et écoutez le résultat.
- Reconstituez le son entier avec la partie contenant le bruit atténué.

Faites de même avec le son `alien_bruit2.wav` (attention, il pourrait y avoir plusieurs fréquences parasites).

Exercice 3.15 Reprenez le signal de l'exercice 2.9. Est-il possible de le filtrer pour supprimer le battement ? Même question pour le signal de l'exercice 2.10.

Exercice 3.16 Créez une suite de notes allant du La de la troisième octave au La de la neuvième octave. Vous aurez sept notes. Choisissez la fréquence d'échantillonnage $F_e = 8000$ Hz.

- Écoutez le résultat et visualisez le spectre du nouveau signal.
- Identifiez les fréquences que vous allez réellement entendre.
- Vérifiez si votre prédiction est correcte.
- Filtrez le signal pour éviter la présence de fréquences fantômes.

Exercice 3.17 Filtrez le signal `tarzan.wav` au moyen d'un filtre de Butterworth passe-bas, d'ordre 2 et de fréquence de coupure $F_c = 1000$ Hz en utilisant les fonctions `signal.butter` et `signal.filtfilt`. Utilisez ensuite les fonctions `fft.fftfreq` et `fft.fft` pour visualiser le spectre avant et après filtrage, comme cela a été fait à la section 3.2.1.

Exercice 3.18 Importez le signal `tarzan.wav`.

- a) Utilisez les fonctions `signal.spectrogram` et `plt.pcolormesh` pour visualiser l'évolution de son spectre au cours du temps.
- b) Créez un signal qui est la somme d'un sinus de 440 Hz et d'un sinus de 800 Hz et qui a le même nombre d'échantillons que le signal sonore. C'est un bruit parasite.
- c) Créez un signal qui est la superposition du signal d'origine et du bruit parasite. Nous l'appelons « signal bruité ».
- d) Visualisez l'évolution de son spectre au cours du temps au moyen d'un spectrogramme.
- e) Réalisez un filtre qui atténue le bruit uniquement sur la partie bruitée. Sauvez ce résultat dans un nouveau fichier `.wav`.

Partie 4

Introduction au traitement d'images

Dans cette partie, nous utiliserons principalement la bibliothèque `scikit-image` spécialisée dans le traitement d'images [2]. Pour l'installer avec pip :

```
python -m pip install -U scikit-image
```

ou avec conda (ou mamba, en remplaçant `conda` par `mamba`) :

```
conda install scikit-image
```

Au début de votre code, importez les sous-packages suivants :

```
from skimage import exposure, color, filters, measure
```

4.1 Ouverture, affichage et types d'images

Exercice 4.1 Créez un dossier `images` dans le dossier contenant votre notebook. Ajoutez-y l'image `moon.tif`. Ensuite, ouvrez celle-ci au moyen de l'instruction :

```
I = plt.imread('images/moon.tif');
```

L'instruction `%whos` indiquera le nom, le type et la taille de l'image. On constate que sa taille est `537x358` et que ses éléments sont du type `'uint8'`. Cela indique qu'elle possède 256 (2^8) niveaux de gris.

Visualisez cette image en niveaux de gris grâce aux instructions :

```
plt.imshow(I, cmap='gray')
plt.show()
```

Placez un rectangle blanc de taille 10×20 dans le coin supérieur gauche de l'image et visualisez le résultat. Sauvez cette nouvelle image en utilisant la fonction `plt.imsave`. Assurez-vous que la colormap de l'image sauvée est en niveaux de gris.

Exercice 4.2 Ouvrez l'image `flowers.tif`.

- Quels sont le type de l'image, le type de ses éléments et la taille de l'image ?
- Sur quatre sous-figures, affichez l'image originale, sa composante rouge (R), sa composante verte (G) et sa composante bleue (B). Est-ce que ceci vous semble cohérent ?
- Utilisez la fonction `measure.profile_line` pour visualiser les composantes R, G et B d'une ligne sur l'image.

Exercice 4.3 Créez et affichez des carrés de taille 400×400 des couleurs suivantes :

- rouge d'intensité maximale ;
- vert d'intensité maximale ;
- bleu d'intensité maximale ;
- rouge et vert d'intensité maximale ;
- rouge et bleu d'intensité maximale ;
- vert et bleu d'intensité maximale ;
- rouge, vert et bleu d'intensité maximale ;
- bleu se trouvant dans le coin inférieur droit de l'image `flowers.tif`.

Exercice 4.4 Ouvrez l'image `flowers.tif`.

- a) Constatez que les éléments de l'image `flowers.tif` sont de type `'uint8'`.
- b) Transformez le type des éléments en `'float64'` grâce à la méthode `astype`. Dans quel intervalle sont compris les niveaux de couleurs ? L'image s'affiche-t-elle correctement ? Si pas, que devrait-on ajouter comme étape pour obtenir le même résultat qu'avec l'image originale en `'uint8'` ? Dans quel intervalle sont maintenant compris les niveaux de couleurs ?

Exercice 4.5 Que signifient contraste, histogramme, brillance (*brightness*) et correction gamma ? Comment corriger une image par ajustement d'intensité ou par égalisation de l'histogramme ? Utilisez la fonction `exposure.equalize_hist` pour améliorer l'image `pout.tif` par correction gamma (testez différentes valeurs pour le paramètre `gamma`).

4.2 Opérations arithmétiques sur une image

Les opérations arithmétiques peuvent être effectuées uniquement sur des images dont les éléments sont de type `float`. Pensez donc à utiliser systématiquement `astype` après l'import de votre image.

Exercice 4.6 Ouvrez l'image `flowers.tif`. Ajoutez la valeur 200 à tous les pixels de l'image et visualisez le résultat. Qu'observez-vous ?

Exercice 4.7 Affichez l'image `flowers.tif` en niveaux de gris sachant que la luminance¹ vaut $0,299 \times R + 0,587 \times G + 0,114 \times B$ où R , G , B sont les matrices représentant les composantes R, G et B de l'image originale. Vous pouvez faire cela manuellement ou à l'aide de la fonction `np.dot`. Comparez votre résultat avec celui obtenu au moyen de la fonction `color.rgb2gray`.

1. Selon la recommandation UIT-R BT 601.

Exercice 4.8 Visualisez l'image `moon.tif` en 3D (pixels `x`, `y` et le niveau de gris) en utilisant les fonctions `np.meshgrid` et `plot_surface` :

```
I = plt.imread('images/moon.tif')
I = I.astype(float)/255
l, c = I.shape
x, y = np.meshgrid(np.arange(c), np.arange(l))

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(x, y, I, cmap='plasma')
plt.show()
```

On peut changer la colormap grâce à l'argument `cmap`.

Exercice 4.9 Ouvrez l'image `moon.tif`. Coloriez en noir chaque pixel de l'image au moyen des trois programmes suivants. Comparez le temps d'exécution de chacun de ces programmes. Vous constatez, une fois de plus, l'intérêt de passer par le calcul vectoriel.

```
I = plt.imread('images/moon.tif')
J = I.copy()

# Programme 1
t1_Start = time.time()
J[:, :] = 0
t1_End = time.time()

print("Temps d'exécution du programme 1 : ", t1_End-t1_Start)

# Programme 2
t2_Start = time.time()
for i in range(J.shape[0]):
    J[i, :] = 0
t2_End = time.time()

print("Temps d'exécution du programme 2 : ", t2_End-t2_Start)

# Programme 3
t3_Start = time.time()
for i in range(J.shape[0]):
    for j in range(J.shape[1]):
        J[i, j] = 0
t3_End = time.time()

print("Temps d'exécution du programme 3 : ", t3_End-t3_Start)
```

Exercice 4.10 L'image `diapo1.jpg` provient d'une ancienne diapositive. Elle est trop rouge. Corrigez-la. Faites de même avec l'image `diapo2.jpg`.

Exercice 4.11 (Filtre moyennneur) Ouvrez l'image `Lenna.bmp`.

- a) Sur l'image, placez des pixels noirs non adjacents et ne touchant pas les bordures, soit à la main soit aléatoirement.
- b) Écrivez un programme qui remplace les points de l'image par la moyenne des 9 points voisins. Visualisez le résultat.
- c) Effectuez une opération similaire au moyen de l'instruction `signal.convolve2d(I, h, mode='same')` où `I` est l'image corrompue par les points noirs et `h` est une matrice de taille 3×3 dont toutes les entrées valent $1/9$. Visualisez le résultat. Que voit-on ?
- d) Recommencez l'opération avec une matrice `h` de taille 5×5 dont les entrées valent $1/25$. Visualisez le résultat. Quelles sont les différences avec celui du point (c) ?
- e) Écrivez un programme qui atténue les points noirs en effectuant une moyenne sur les 8 points voisins (en ne prenant pas en compte la valeur centrale, qui est celle du pixel noir). Visualisez le résultat. Est-il meilleur ? À quelle matrice `h` correspond cette opération ?
- f) Essayez d'améliorer ce filtre, par exemple en ne l'appliquant qu'aux points noirs « isolés », c'est-à-dire ceux pour lesquels la différence de valeur avec ses voisins dépasse un certain seuil.
- g) Appliquez un filtre médian (`signal.medfilt2d`) à l'image corrompue initiale et comparez le résultat avec celui obtenu via le filtre moyenneur.

Exercice 4.12 (Détection de bords) Un bord (*edge* en anglais) est un chemin délimitant des zones de l'image dont les intensités sont très différentes. Un bord correspond souvent à la frontière d'un objet.

Ouvrez l'image `moon.tif` et sauvez-la dans la variable `I`. Le but de cet exercice est de tester différentes méthodes de détection des bords.

- a) Écrivez un programme qui remplace chaque pixel de l'image par la valeur du pixel de gauche moins la valeur du pixel du dessous. Affichez le résultat. Donnez la valeur de la matrice `h` auquel ce filtre correspond.
- b) Affichez les bords des objets après avoir filtré l'image avec les instructions suivantes :

```
I_sobel = filters.sobel(I)
I_prewitt = filters.prewitt(I)
```

Constatez les différences entre ces deux résultats.

- c) La matrice `h` correspondant au filtre de Sobel est

$$h = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}.$$

On constate que, pour le filtrage d'un pixel, seuls les pixels des lignes supérieure et inférieure sont pris en compte. Si le pixel à remplacer est le pixel de valeur I_{ij}

(où i représente l'indice de la ligne et j , l'indice de la colonne), donnez la formule de la valeur de ce pixel après filtrage.

- d) Essayez l'instruction `signal.convolve2d(I, h, mode='same')` et interprétez les résultats obtenus.

4.3 Opérations sur une image binaire

La fonction `measure.label` permet de réaliser un nombre très important d'opérations sur les images binaires via la « labelisation » de l'image. Cette opération consiste à repérer les objets non connexes de l'image puis à numéroter tous les pixels de chaque objet par un même nombre. En comptant le nombre de pixels par objet, on peut en déduire sa surface. On peut également avoir accès à d'autres propriétés de ceux-ci : centre de gravité, excentricité, etc. La fonction `measure.regionprops` donne accès à ces propriétés. Voici un exemple de son utilisation sur une image binaire labellisée `L` :

```
regions = measure.regionprops(L)

eccentricity_L = [props.eccentricity for props in regions]
area_L        = [props.area       for props in regions]
```

Exercice 4.13 Ouvrez l'image `albireo.bmp`.

- Transformez cette image en image binaire au moyen de l'instruction `color.rgb2gray` suivie d'un seuil manuel (> 0.5). Affichez le résultat et constatez qu'un des objets a disparu.
- Résolvez le problème précédent en choisissant un seuil adéquat (utilisez la fonction `filters.threshold_otsu`).
- Définissez les objets présents et leur nombre au moyen de la fonction `measure.label`. Vérifiez que deux objets sont bien identifiés. La fonction `measure.label` attribue à tous les pixels d'un même objet un même nombre entier (qui commence à 1).
- Utilisez la fonction `measure.regionprops` pour accéder aux propriétés des objets. Celle-ci vous renverra une liste `regions`. `[props.area for props in regions]` renvoie un vecteur contenant la surface de chaque objet (en nombre de pixels).
- Identifiez automatiquement le label de l'objet ayant la plus grande surface.
- Affichez uniquement cet objet.
- Affichez côte à côte l'image initiale, l'image binaire correspondante et l'image finale. Vérifiez visuellement que celle-ci correspond bien à la condition demandée.

Exercice 4.14 Ouvrez l'image `circuit.tif` et constatez que l'image comporte de petites taches. Utilisez la fonction `measure.label` pour parvenir à les éliminer.

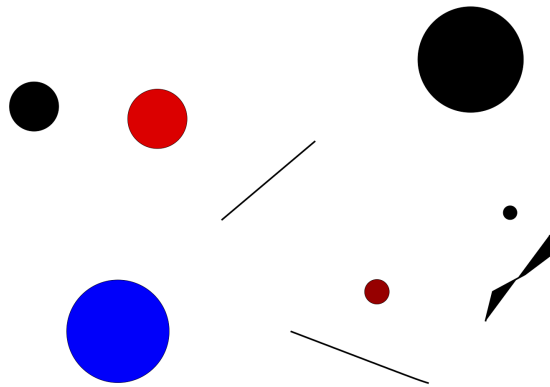


Figure 4.1 – Image de l'exercice 4.15.

Exercice 4.15 Considérons l'image `label_exc.jpg` (voir Figure 4.1). On souhaite détecter les objets allongés de l'image. Procédez de la même manière que dans l'exercice 4.13 mais utilisez plutôt la propriété d'excentricité (`eccentricity`). Il faut également inverser l'image binaire avant de la labeliser. Ne pas le faire impliquerait de n'avoir qu'un seul objet : le *background*.

Partie 6

Vademecum Python

Ce vademecum n'est pas exhaustif : n'hésitez pas à y ajouter les fonctions supplémentaires que vous trouvez intéressantes.

6.1 Introduction au calcul matriciel avec Python

Les commandes de cette section se rapportent à la partie 1, page 7.

6.1.1 Prise en main

- `#` : délimiteur de commentaires.
- `print()` : pour d'afficher la valeur des variables.
- `help(commande)` : affiche l'aide de la commande.
- `s.dtype` : affiche le type des données contenues dans `s` (e.g., `numpy.uint8`).
- `%whos` : affiche la liste des variables de l'espace de travail.
- `time.time()` : associé à des variables, cela permet de connaître le temps d'exécution d'un programme.
- `abs(z)` : module de `z`.
- `numpy.angle(z)` : argument/phase de `z` en radians.
- `z.conjugate()` : complexe conjugué de `z`.
- `numpy.rad2deg(a)` ou `numpy.degrees(a)` : transforme en degrés la quantité `a` exprimée en radians.
- `numpy.deg2rad(a)` ou `numpy.radians(a)` : transforme en radians la quantité `a` exprimée en degrés.

6.1.2 Vecteurs et matrices

- `t = numpy.arange(1, 10.1, 0.1)` : définit un vecteur `t` composé de nombres entre 1 et 10 avec un pas de 0,1 (par défaut, la valeur du pas est 1).
- `t = numpy.linspace(1, 10, 100)` : définit un vecteur `t` composé de 100 nombres répartis uniformément entre 1 et 10.
- `a = numpy.array([1, 3, 5, 2, -1])` : définit un vecteur ligne `a` contenant les éléments 1, 3, 5, 2 et -1.
- `a = numpy.array([[1], [3], [5], [2], [-1]])` : définit un vecteur colonne `a` contenant les éléments 1, 3, 5, 2 et -1.
- `m = numpy.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])` : définit la matrice `m` de dimension 3×3 :

$$m = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

- `c = numpy.array([a, b])` : concatène des vecteurs `a` et `b`.
- `c = a.flatten()` : retourne l'array `a` « aplati » selon une dimension.
- `t[1]` : permet d'accéder au deuxième élément de l'array `t`.
- `t[-1]` : permet d'accéder au dernier élément de l'array `t`.
- `t[:5]` : permet d'accéder aux cinq premiers éléments de l'array `t`.
- `t[1:4]` : renvoie du deuxième au quatrième élément de l'array `t`.
- `m[1, 2]` : permet d'accéder à l'élément de la seconde ligne et de la troisième colonne de la matrice `m`.
- `m[1, :]` : renvoie la deuxième ligne de la matrice `m`.
- `m[:, 2]` : renvoie la troisième colonne de la matrice `m` sous forme d'un vecteur ligne.
- `len(m)` : renvoie le nombre d'éléments de l'array `m`.
- `numpy.size(m)` : renvoie le nombre d'éléments de l'array `m`, possiblement selon un axe donné.
- `numpy.shape(m)` : renvoie les dimensions de l'array `m`, c'est-à-dire son nombre de lignes et son nombre de colonnes.
- `numpy.min(m)/numpy.max(m)` : renvoie la valeur minimum/maximum de l'array `m`.
- `numpy.mean(m)` : renvoie la valeur moyenne des entrées de l'array `m`.

Importation de signaux audio

- `Fs, s = scipy.io.wavfile.read('filename.wav')` : ouvre le fichier `filename.wav` et stocke les échantillons dans le vecteur `s`. `Fs` contient la fréquence d'échantillonnage du signal.
- `scipy.io.wavfile.write('filename.wav', Fs, data)` : enregistre les données du vecteur `data` dans le fichier `filename.wav`. `Fs` contient la fréquence d'échantillonnage du signal.
- `sounddevice.play(s, Fs)` : joue le signal `s` avec une fréquence d'échantillonnage `Fs`.
- `sounddevice.wait()` : attend que le son joué soit terminé avant d'exécuter la suite du programme.

6.1.3 Programmation avec Python

- `for i in range(1, 11): <instructions>` : boucle `for` (remarque : attention aux performances par rapport au calcul matriciel et attention à l'indentation des instructions).
- `while condition: <instructions>` : boucle `while`.
- `if condition1: <instructions> elif condition2: <instructions> else: <instructions>` : condition.
- `<, <=, >, >=, ==, !=` : opérateurs de comparaison.
- `and, or, not` : opérateurs logiques.

6.1.4 Éléments de calcul matriciel

- `numpy.zeros((m, n))` : envoie une matrice de dimension $m \times n$ composée de 0 partout.
- `numpy.ones((m, n))` : envoie une matrice de dimension $m \times n$ composée de 1 partout.
- `numpy.eye(m, n)` : envoie une matrice de dimension $m \times n$ composée de 0 à l'exception de la diagonale principale qui contient des 1.
- `A @ B` ou `numpy.matmul(A, B)` : renvoie le produit matriciel de A par B (fonctionne uniquement si le nombre de lignes de A est égal au nombre de colonnes de B).
- `A * B` : renvoie le produit terme par terme de A par B (fonctionne uniquement si A et B sont de mêmes dimensions).
- `A.T` : renvoie la transposée de la matrice A.
- `numpy.linalg.inv(A)` : renvoie la matrice inverse de la matrice A si elle existe.
- `numpy.linalg.matrix_power(A, 2)` : renvoie le carré de la matrice A (donc `A @ A`).
- `numpy.square(A)` : renvoie une matrice dans laquelle chaque élément de A a été mis au carré (donc `A * A`).
- `numpy.random.rand(s1, s2)` : génère un array de dimension $s1 \times s2$ peuplé de nombres aléatoires générés à partir d'une distribution uniforme sur $[0, 1[$.
- `numpy.random.randint(low, high, (s1, s2))` : génère un array de dimension $s1 \times s2$ peuplé de nombres aléatoires générés à partir d'une distribution uniforme discrète sur $[low, high[$.
- `numpy.random.randn(s1, s2)` : génère un array de dimension $s1 \times s2$ peuplé de nombres aléatoires générés à partir d'une distribution normale standard.
- `numpy.diag(A)` : renvoie les éléments de la diagonale de la matrice A.

6.2 Représentation des signaux

Les commandes de cette section se rapportent à la première section de la partie 2, page 13.

- `matplotlib.pyplot.figure()` : crée une nouvelle fenêtre/figure.
- `matplotlib.pyplot.show()` : afficher toutes les figures ouvertes.
- `matplotlib.pyplot.close('all')` : ferme toutes les fenêtres ouvertes.
- `matplotlib.pyplot.title('Titre')` : ajoute un titre au graphe.
- `matplotlib.pyplot.xlabel("Nom de l'axe x")` ou `matplotlib.pyplot.ylabel("Nom de l'axe y")` : ajoute un titre à l'axe correspondant.
- `matplotlib.pyplot.xlim((p, q))` : permet de limiter les valeurs affichées en abscisses entre p et q (à préciser avant le `matplotlib.pyplot.show()`).
- `matplotlib.pyplot.tight_layout()` : permet d'ajuster la disposition du graphique (à préciser avant le `matplotlib.pyplot.show()`).
- `matplotlib.pyplot.subplot(a, b, c)` : permet d'afficher plusieurs figures dans la même fenêtre. Les paramètres a et b définissent le nombre de figures en lignes et en colonnes et c spécifie la position du graphe créé par la commande.

- `fig, axs = matplotlib.pyplot.subplots(a, b)` : crée une figure avec plusieurs axes (un par sous-figure). Les paramètres `a` et `b` définissent le nombre d'axes en lignes et en colonnes. D'autres paramètres additionnels permettent par exemple de partager une même graduation d'axe pour toutes les sous-figures.

Signaux 1D

- `matplotlib.pyplot.plot(t, y)` : affiche le graphe du vecteur `y` en fonction du vecteur `t`.
- `matplotlib.pyplot.stem(x, y)` : permet de dessiner uniquement les points de coordonnées `(x, y)` sans les relier (à la manière d'un graphe discret).

Signaux 2D

- `matplotlib.pyplot.imshow(I, cmap='gray')` : affiche dans une figure l'image représentée par la matrice `I` en niveaux de gris. La carte des couleurs par défaut est `'viridis'`.
- `matplotlib.pyplot.set_cmap('map')` : permet de changer la carte des couleurs courante (valeurs possibles : `'viridis'`, `'jet'`, `'hot'`, `'cool'`, `'summer'`, `'hsv'`, `'gray'`, `'Pastell1'`, etc.).
- `matplotlib.pyplot.colorbar()` : affiche une barre représentant les niveaux de couleurs de la colormap de l'image représentée dans la figure courante.

6.3 Représentation fréquentielle des signaux 1D

Les commandes de cette section se rapportent à la partie 3, page 19.

- `numpy.fft.fft(s)` : renvoie la transformée de Fourier discrète du signal `s`.
- `numpy.fft.fftfreq(len(t), 1/Fs)` : renvoie les fréquences auxquelles correspondent les valeurs de la FFT d'un signal à `len(t)` points et de fréquence d'échantillonnage `Fs`.
- `numpy.fft.ifft(X)` : calcule la transformée de Fourier inverse du spectre `X`.
- `b, a = scipy.signal.butter(n, Wn, 'ftype')` : calcule les coefficients d'un filtre de Butterworth d'ordre `n` avec une fréquence de coupure normalisée `Wn` ($W_n = \frac{F_c}{F_s/2}$, avec F_c la fréquence de coupure en Hz). `ftype` est le type du filtre (passe-bas par défaut, les valeurs spécifiées sont `'high'`, `'low'`, `'stop'` et `'pass'`).
- `scipy.signal.filtfilt(b, a, s)` : filtre le signal `s` avec un filtre de coefficients `a, b`.
- `f, t, Sxx = scipy.signal.spectrogram(s, Fs, window='hamming', nperseg=1024, noverlap=512)` : calcule le spectrogramme du signal `s`. `Sxx` est la matrice du spectrogramme de dimensions données par les longueurs des vecteurs `f` et `t`. `window` est la forme du fenêtrage temporel, `nperseg` est la taille des segments du signal sur lesquels sera calculée la FFT. `noverlap` donne le nombre

d'échantillons de superposition entre deux positions successives de la fenêtre glissante (ici 50%). Finalement, `Fs` est la fréquence d'échantillonnage du signal. Remarque : on affiche le spectrogramme sous la forme d'un nuage de points colorés avec l'instruction `matplotlib.pyplot.pcolormesh(t, f, np.log10(Sxx))`. L'affichage logarithmique pour améliorer la visualisation des valeurs faibles du spectrogramme.

6.4 Traitement d'images

Les commandes de cette section se rapportent à la partie 4, page 25.

6.4.1 Ouverture, affichage et types d'images

- `matplotlib.pyplot.imread('filename')` : ouvre le fichier `filename` (par exemple `.tif`, `.jpg`, `.bmp`, etc.) et renvoie soit une matrice bidimensionnelle (image en niveaux de gris) soit une matrice tridimensionnelle (image en couleurs).
- `I.copy()` : renvoie une copie de l'array `I`.
- `I.astype(np.float64)` : permet de convertir les données de `I` dans un autre type (ici vers des nombres à virgule flottante).
- `skimage.measure.profile_line(I, point_start, point_end)` : renvoie un tableau contenant les valeurs RGB des pixels sur une ligne allant de `point_start` à `point_end` dans l'image `I`.

6.4.2 Opérations arithmétiques sur une image

- `skimage.color.rgb2gray(I)` : convertit l'image RGB `I` en niveaux de gris en utilisant la formule `gray = 0.2125*R + 0.7154*G + 0.0721*B`.
- `skimage.exposure.equalize_hist(I)` : égalise l'histogramme de l'image `I` (*i.e.*, répartit les valeurs des pixels sur toute la gamme disponible pour améliorer le contraste) et renvoie l'image résultante.
- `skimage.exposure.rescale_intensity(I)` : ajuste l'intensité ou la carte de couleurs de l'image `I` et renvoie l'image résultante.
- `skimage.filters.sobel(I)` : renvoie une image contenant les contours de l'image `I` obtenus grâce au filtre de Sobel. D'autres filtres peuvent être utilisés pour détecter les contours, *e.g.*, `skimage.filters.prewitt(I)` ou `skimage.filters.roberts(I)`.
- `scipy.signal.convolve2d(I, h, mode='same')` : renvoie le produit de convolution des deux arrays bidimensionnels `I` et `h`. Le paramètre `mode='same'` fait en sorte que le résultat a la même taille que `I`.
- `scipy.signal.correlate2d(I, h, mode='same')` : renvoie la corrélation des deux arrays bidimensionnels `I` et `h`. Le paramètre `mode='same'` fait en sorte que le résultat a la même taille que `I`.
- `scipy.signal.medfilt2d(I, kernel_size=[M, N])` : renvoie l'image `I` modifiée suite à l'application d'un filtre médian dont le noyau est de taille $M \times N$ (avec `M` et `N` impairs). La taille du noyau par défaut est 3×3 .

6.4.3 Opérations sur une image binaire

Binarisation et labellisation

- `I >= thresh` : transforme l'image `I` en image binaire avec un seuil `thresh`.
- `skimage.filters.threshold_otsu(I)` : renvoie un seuil calculé automatiquement sur base de l'image `I` et selon la méthode d'Otsu.
- `L, n = skimage.measure.label(I, return_num=True)` : identifie les objets (régions connectées) présents dans l'image binaire `I`. La matrice `L` a la même taille que `I` et contient des valeurs entières. Tous les pixels appartenant au $i^{\text{ème}}$ objet détecté ont la valeur i . La paramètre `return_num=True` permet de renvoyer également le nombre d'objets détectés.
- `skimage.segmentation.find_boundaries(L)` : renvoie une image booléenne basée sur l'image labellisée `L` et contenant `True` sur les frontières entre régions.
- `skimage.measure.regionprops(L)` : mesure une série de propriétés des objets présents dans une image labellisée `L`. La fonction renvoie un tableau contenant autant de structures que d'objets. Chaque champ d'une structure correspond à une propriété (par exemple, `data[1].area` contient la surface du deuxième objet).
- `numpy.where(condition, x, y)` : renvoie `x` ou `y` dépendant si `condition` est `True` ou `False`. Les deux derniers arguments sont optionnels. Par exemple, `numpy.where(L==2, 1, 0)` renvoie une image binaire contenant uniquement le deuxième objet de l'image labellisée `L`.
- `numpy.isin(X, a)` : renvoie une matrice binaire de même taille que `X` contenant des valeurs `True` si la valeur correspondante de `X` est présente dans le array `a` et `False` dans le cas contraire.

Opérations morphologiques

- `skimage.morphology.diamond(s)` : crée un élément structurant pour les opérations d'érosion et de dilatation. Ici, il s'agit d'une forme en diamant de taille `s` (entier) contrôlant la taille du losange. D'autres formes sont possibles, *e.g.*, un disque avec `skimage.morphology.disk(s)` ou un carré avec `skimage.morphology.square(s)`.
- `skimage.morphology.erosion(I, SE)` : renvoie une nouvelle image sur base de l'image `I` (en niveaux de gris ou binaire) à laquelle l'opération d'érosion a été appliquée avec un élément structurant `SE`.
- `skimage.morphology.dilation(I, SE)` : renvoie une nouvelle image sur base de l'image `I` (en niveaux de gris ou binaire) à laquelle l'opération de dilatation a été appliquée avec un élément structurant `SE`.
- `skimage.morphology.remove_small_objects(I, min_size=10)` : renvoie une image binaire identique à l'image de départ `I` à l'exception des régions connectées plus petites que `min_size` pixels qui sont supprimées.
- `skimage.morphology.skeletonize(I)` : renvoie une image contenant le squelette de l'image binaire `I`.
- `scipy.ndimage.binary_fill_holes(I)` : remplit les trous de l'image binaire `I`.

Liens vers les documentations en ligne

Les documentations complètes se trouvent aux adresses suivantes :

- Python : <https://docs.python.org>
- Matplotlib : <https://matplotlib.org/>
- NumPy : <https://numpy.org/>
- scikit-image : <https://scikit-image.org/>
- SciPy : <https://scipy.org/>
- sounddevice : <https://python-sounddevice.readthedocs.io/>
- time : <https://docs.python.org/3/library/time.html>
- OpenCV : <https://docs.opencv.org/4.x/>

Thème III

Introduction au *machine learning*

Partie 7

Clustering

En *machine learning* (ML), le *clustering* est la tâche qui consiste à *regrouper* des données qui sont similaires. On cherche à identifier des groupes (*clusters*), à diviser l'espace en différentes zones. Les points centraux (centres de masse) de ces zones sont appelés *centroïdes*. Ce sont de bons représentants des propriétés moyennes de leur groupe.

Le *clustering* fait partie des **apprentissages non-supervisés** : on ne connaît pas d'avance à quelle catégorie appartient chacune des données.

7.1 Jeu de données : les iris de Fisher (1936)

Ce jeu de données¹ largement utilisé contient 150 observations réalisées sur trois espèces d'iris (*iris setosa*, *iris versicolor* et *iris virginica*). Chaque observation possède quatre caractéristiques et correspond donc à un vecteur à quatre composantes : largeur et longueur du pétale ainsi que largeur et longueur du sépale (voir Figure 7.1).

On a également accès à la classe de chaque observation. On n'utilise cependant pas cette information pour former les *clusters* (apprentissage non-supervisé) mais plutôt pour comparer les résultats des algorithmes aux groupes d'iris réels.



Figure 7.1 – Fleur d'iris. Source : <https://kidiscience.cafe-sciences.org/articles/predire-les-longueurs-de-petale-des-fleurs-diris/>.

La librairie `sklearn` inclut plusieurs jeux de données dont celui des iris :

```
import scipy
import sklearn
from sklearn import datasets
```

Exécutez le code suivant pour importer les données et visualiser les noms des caractéristiques et des espèces ainsi que la classe (label) de chaque observation.

1. Plus d'informations ici : https://fr.wikipedia.org/wiki/Iris_de_Fisher.

```
iris = datasets.load_iris()

print(dir(iris))          # contenu de la structure
print(iris.feature_names) # noms des caractéristiques des observations
print(iris.target_names)  # labels de classification
print(iris.target)        # numéro de l'espèce à laquelle appartient chaque
                          # observation
```

Exercice 7.1 Visualisez sur une figure les 150 observations sur base de la taille des pétales (longueur des pétales en abscisses et largeur en ordonnées). On pourrait faire pareil sur base de deux autres des quatre caractéristiques des vecteurs.

```
import matplotlib.pyplot as plt
import numpy as np

# Visualisation des données
fig = plt.figure(figsize=(10, 6))
ax1 = plt.subplot(1,1,1)
ax1.scatter(iris.data[:,2], iris.data[:,3])
plt.xlabel('Longueur du pétale (cm)')
plt.ylabel('Largeur du pétale (cm)')
plt.title("Observation d'iris")

plt.show()
```

7.2 Les ingrédients du *clustering*

Pour pouvoir réaliser du *clustering*, on a besoin de (i) un ensemble de k centroïdes qui définissent les centres de k groupes et (ii) une fonction associant chaque observation au numéro du centroïde le plus proche.

Les centroïdes initiaux peuvent par exemple être choisis aléatoirement parmi les données.

7.3 Algorithme k -means

L'algorithme k -means est un des algorithmes de *clustering* les plus connus et utilisés. Il contient les étapes suivantes :

1. Initialisation des centroïdes.
2. Association de chaque donnée au centroïde le plus proche.
3. Mise à jour des centroïdes (moyenne des observations appartenant au groupe).
4. Vérification du critère d'arrêt.
5. Si le critère d'arrêt n'est pas satisfait, retour à l'étape 2.

Dans le cas du jeu de données des iris, $k = 3$ car il y a trois espèces de fleurs. En général, le nombre de groupes n'est pas connu à l'avance.

Exercice 7.2 Importez l'algorithme `KMeans` de la librairie `sklearn.cluster` et construisez un modèle `model` avec trois clusters. Appliquez ensuite l'algorithme aux données `iris.data` (allez voir la documentation). Les groupes identifiés pour chaque observation sont stockés dans `model.labels_` et les coordonnées des centroïdes dans `model.cluster_centers_`.

Pour rendre le résultat plus lisible, on peut plutôt afficher les *clusters* sur des graphiques (une couleur par espèce) :

```
# Visualisation des données initiales et de leur groupe
colormap = np.array(['c', 'm', 'y'])

fig = plt.figure(figsize=(10, 6))
ax1 = plt.subplot(2,1,1)
ax1.scatter(iris.data[:,2], iris.data[:,3], c=colormap[iris.target])
plt.xlabel("Longueur du pétale (cm)")
plt.ylabel("Largeur du pétale (cm)")
plt.title("Groupes réels dans le jeu de données")

# Visualisation des données et des centroïdes après convergence de l'algorithme
ax2 = plt.subplot(2,1,2)
ax2.scatter(iris.data[:,2], iris.data[:,3], c=colormap[model.labels_])
ax2.scatter(model.cluster_centers_[:,2], model.cluster_centers_[:,3], s=200, c=
            ['c', 'm', 'y'])
plt.xlabel("Longueur du pétale (cm)")
plt.ylabel("Largeur du pétale (cm)")
plt.title("Observation des données quantifiées et centroïdes après convergence
            de l'algorithme")

plt.show()
```

Exercice 7.3 Affichez des figures similaires à celles générées précédemment mais montrant la largeur des sépales en fonction de leur longueur.

Pour information. L'algorithme converge mais n'atteint pas forcément le minimum global (souvent, il atteint même plutôt un minimum *local*). Cela dépend des conditions initiales de l'algorithme (le choix des centroïdes initiaux). Une pratique courante est d'exécuter plusieurs fois l'algorithme *k-means* (par exemple, 10 fois) et de considérer l'occurrence qui mène à la plus petite erreur (`model.inertia_`).

Partie 8

Classification binaire

En ML, la *classification* est la tâche qui consiste à *labelliser* les données ou à les classer en *catégories*. Dans un premier temps, nous considérons la classification *binaire* : l'output peut prendre uniquement deux valeurs : +1 (qui correspond à **True**) ou 0 (qui correspond à **False**).

La classification fait partie des **apprentissages supervisés** : on construit un classificateur à partir d'un jeu de données pour lesquelles on connaît les labels (phase d'entraînement). Le classificateur est ensuite appliqué dans un second temps à des données qui n'ont pas servi à le construire afin d'évaluer ses performances (phase de validation).

8.1 Jeu de données : cancer du sein

Ce jeu de données contient 569 observations de tumeurs qui sont soit malignes soit bénignes. Chaque observation possède 30 caractéristiques et correspond à un vecteur à 30 composantes. Les caractéristiques sont liées à la morphologie de la tumeur (périmètre, aire, compacité, symétrie, etc.).

On a également l'information, pour chaque observation, de la nature de la tumeur : maligne (0) ou bénigne (1). On utilisera cette information lors de la phase d'entraînement pour construire le classificateur.

On utilise à nouveau la librairie `sklearn` pour importer le jeu de données et afficher les noms des caractéristiques :

```
import numpy as np
import scipy
import sklearn
from sklearn import datasets
data_tumor = datasets.load_breast_cancer()

print(dir(data_tumor))          # contenu de la structure
print(data_tumor.feature_names) # noms des caractéristiques des observations
print(data_tumor.target_names)  # labels de classification
print(data_tumor.target)        # nature de la tumeur pour chaque observation
```

Exercice 8.1 Visualisez sur une figure les 569 observations en mettant en abscisses le périmètre moyen (`mean perimeter`) et en ordonnées, la texture moyenne (`mean texture`). Veillez à ce que les observations de tumeurs malignes soient dans une couleur et celles de tumeurs bénignes dans une autre. *Indication : basez-vous sur le code de l'exercice 7.1.*

8.2 Les ingrédients de la classification

Pour pouvoir réaliser de la classification binaire, on a besoin d'une fonction qui associe à un vecteur d'observations un label 0 ou 1. Cette fonction est appelée le *classificateur*.

On constate sur le graphique que vous avez produit qu'en première approche, une droite adaptée pourrait servir à discriminer les tumeurs malignes des tumeurs bénignes. On appelle celle-ci une *droite de régression linéaire*. Le classificateur est une fonction qui va attribuer un label en fonction de la position de la donnée par rapport à la droite (un côté ou l'autre).

Exercice 8.2 De la librairie `sklearn.linear_model`, importez la classe `LinearRegression` et construisez un objet de régression linéaire. Entraînez-le ensuite avec les données médicales grâce à la fonction `model.fit`. N'hésitez pas à aller voir la documentation.

On peut évaluer les performances d'un modèle entraîné en lui demandant de classer des nouvelles données. Pour cela, on sépare le jeu de données en deux groupes :

- un groupe avec 80% des données pour *entraîner* le modèle de régression linéaire (ou autre) ;
- un groupe avec les 20% restants pour *valider* le modèle sur des nouvelles données.

Exécutez le code suivant pour séparer le jeu de données initial.

```
from sklearn.model_selection import train_test_split

# Séparation du jeu de données : set d'entraînement et set de validation
X = data_tumor.data
Y = data_tumor.target
X_train, X_val, Y_train, Y_val = train_test_split(X, Y, test_size=0.20,
                                                random_state=1)
```

Pour établir les performances du classificateur, on compare les labels réels y du set de validation à ceux prédits par le classificateur, $\hat{y}$. Il y a quatre options possibles :

- Vrai positif : $y = 1$ et $\hat{y} = 1$
- Vrai négatif : $y = 0$ et $\hat{y} = 0$
- Faux positif : $y = 0$ et $\hat{y} = 1$
- Faux négatif : $y = 1$ et $\hat{y} = 0$

La *matrice de confusion* C contient le nombre de vrais positifs (N_{vp}), de vrais négatifs (N_{vn}), de faux positifs (N_{fp}) et de faux négatifs (N_{fn}) obtenus avec le classificateur,

$$C = \begin{bmatrix} N_{vp} & N_{fn} \\ N_{fp} & N_{vn} \end{bmatrix}.$$

Exercice 8.3 Entraînez à nouveau votre modèle mais uniquement sur le set d'entraînement. Appliquez le classificateur au set de validation pour obtenir les labels prédits `predictions`. Construisez la matrice de confusion associée grâce à la commande :

```
from sklearn.metrics import confusion_matrix

print('Matrice de confusion :')
print(confusion_matrix(Y_val, predictions))
```

Exercice 8.4 Vous pouvez également évaluer la *précision* p ($0 \leq p \leq 1$) du modèle,

$$p = \frac{N_{vp} + N_{vn}}{N},$$

où N est le nombre d'observations dans le set de validation. C'est également faisable en exécutant :

```
from sklearn.metrics import accuracy_score

print('Précision de la régression linéaire :')
print(-accuracy_score(Y_val, predictions))
```

Exercice 8.5 Visualisez sur deux sous-figures similaires à l'exercice 8.1 les données du set de validation avec les vrais labels (gauche) et les labels prédits (droite).

8.3 Comparaison de modèles

Plusieurs méthodes d'apprentissage supervisé sont disponibles avec `sklearn`. On aimerait savoir laquelle est la plus adaptée pour le type de données que l'on traite. Pour cela, on évalue les performances de chaque modèle sur le set de validation. Exécutez le code suivant pour ajouter les différents modèles dans une liste.

```
# Instanciation des algorithmes
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

models = []
models.append(('LiR', LinearRegression))
models.append(('LoR', LogisticRegression(solver='liblinear', multi_class='ovr')))
models.append(('LDA', LinearDiscriminantAnalysis()))
models.append(('KNN', KNeighborsClassifier()))
models.append(('CART', DecisionTreeClassifier()))
models.append(('NB', GaussianNB()))
models.append(('SVM', SVC(gamma='auto')))
```

Exercice 8.6 En effectuant une boucle qui parcourt tous les éléments de `models`, entraînez chacun des modèles sur le set d'entraînement, appliquez le modèle au set de validation et calculez la performance du modèle. Affichez ensuite les performances de chaque modèle. Lequel a les meilleurs performances dans votre cas ?

Remarque : validation croisée. En général, on ne divise pas le jeu de données en deux groupes mais en plusieurs *folds* (par exemple, 10). On fait alors de la *validation croisée* : s'il y a 10 folds, on entraîne 10 fois le modèle sur 9 des folds et on le valide avec le dixième (qui varie pour chacun des entraînements). La précision obtenue (moyenne des 10 précisions) est meilleure que dans le cas implémenté ci-dessus. Les quelques lignes ci-après permettent de faire cela :

```
from sklearn.model_selection import cross_val_score, StratifiedKFold

kfold = StratifiedKFold(n_splits=10, random_state=1, shuffle=True)
cv_results = cross_val_score(model, X_train, Y_train, cv=kfold, scoring='
    accuracy')
```

Classification multi-classes. Ce que vous avez vu ici est également valable lorsque vous travaillez avec plusieurs classes dont les labels sont 1, 2, 3...

Références

- [1] Edward W Kamen and Bonnie S Heck. *Fundamentals of signals and systems using the web and matlab*. Prentice-Hall, Inc., 2006.
- [2] Stéfan van der Walt, Johannes L. Schönberger, Juan Nunez-Iglesias, François Boulgogne, Joshua D. Warner, Neil Yager, Emmanuelle Gouillart, Tony Yu, and the scikit-image contributors. scikit-image : image processing in Python. *PeerJ*, 2 :e453, 6 2014.